

Credit Card Fraud Detection

Academic Machine Learning Pipeline

Project Technical Report

March 2026

Models Implemented

Logistic Regression (Full-Batch GD) | Neural Network (SGD) | XGBoost

AUC: 0.963 | 0.963 | 0.946

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement & Dataset	5
2.1 Dataset Overview	5
2.2 Challenges	5
3. Exploratory Data Analysis (EDA)	6
3.1 Class Distribution & Feature Correlations	6
3.2 Selected Features	6
3.3 Correlation Heatmap of Selected Features	7
4. Feature Engineering — Toeplitz Sliding Window	8
4.1 Mathematical Motivation	8
4.2 Implementation	8
4.3 Why This Helps	8
5. Data Preprocessing & Balancing	10
5.1 Train/Test Split	10
5.2 Class Imbalance Handling — Undersampling	10
5.3 Feature Scaling	10
6. Model Architectures & Training	11
6.1 Logistic Regression — Full-Batch Gradient Descent	11
Theory	11
Hyperparameters	11
6.2 Neural Network — Stochastic Gradient Descent	11
Architecture	11
Backpropagation	11
Hyperparameters	11
6.3 XGBoost — Ensemble Baseline	12
7. Training Dynamics	13
7.1 Full-Batch GD Observations	13
7.2 SGD (Neural Network) Observations	13
8. Results & Evaluation	14
8.1 ROC Curves	14
8.2 Quantitative Comparison	14
8.3 Analysis of Results	15
9. Pipeline Design & Code Architecture	16
9.1 Key Implementation Details	16

10. Conclusions & Future Work..... 17

10.1 Conclusions..... 17

10.2 Limitations 17

10.3 Future Work..... 17

1. Executive Summary

This report presents a complete end-to-end machine learning pipeline for credit card fraud detection. The pipeline addresses the key challenges inherent in financial fraud data: extreme class imbalance (150,337 legitimate vs. 294 fraud transactions), high-dimensional PCA-transformed features, and the need for high-sensitivity detection to minimise missed fraud cases.

Three models were implemented, trained, and compared:

- Logistic Regression trained with full-batch Gradient Descent (GD) — **ROC-AUC: 0.963**
- Neural Network (one hidden layer, ReLU) trained with true SGD — **ROC-AUC: 0.963**
- XGBoost ensemble baseline — **ROC-AUC: 0.946**

A novel Toeplitz / sliding-window feature engineering step was also incorporated to capture temporal patterns in transaction sequences. The pipeline was designed for Google Colab compatibility with inline visualisation and reproducibility.

Key Finding: Both the custom Logistic Regression and the Neural Network achieved an equal best AUC of 0.963, outperforming the XGBoost baseline (0.946). This demonstrates that from-scratch implementations with proper class balancing can compete with gradient-boosted ensembles on structured fraud data.

2. Problem Statement & Dataset

2.1 Dataset Overview

The dataset is the standard ULB Credit Card Fraud Detection dataset (creditcard.csv). It contains European cardholders' transactions over two days in September 2013.

- Total transactions: 150,631
- Legitimate (Class 0): 150,337 — 99.8% of all records
- Fraudulent (Class 1): 294 — 0.195% of all records
- Features: 30 input features — Time, Amount, and V1–V28 (PCA-transformed for privacy)
- Target: Binary Class label (0 = legitimate, 1 = fraud)

2.2 Challenges

- Severe class imbalance: The 512:1 ratio means naive models predict all-legitimate and still achieve >99% accuracy, making accuracy a meaningless metric.
- PCA-transformed features: V1–V28 are anonymised principal components; direct domain interpretation is not possible.
- Fraud pattern subtlety: Fraudulent transactions often resemble legitimate ones in raw feature space, requiring sophisticated models and careful feature selection.
- Evaluation metric: ROC-AUC and Recall (sensitivity) are the appropriate metrics since missing a fraud is far costlier than a false alarm.

3. Exploratory Data Analysis (EDA)

3.1 Class Distribution & Feature Correlations

The EDA phase produced two key visualisations: a bar chart showing the extreme class imbalance and a bar chart showing the Pearson correlation of each feature with the target Class column.

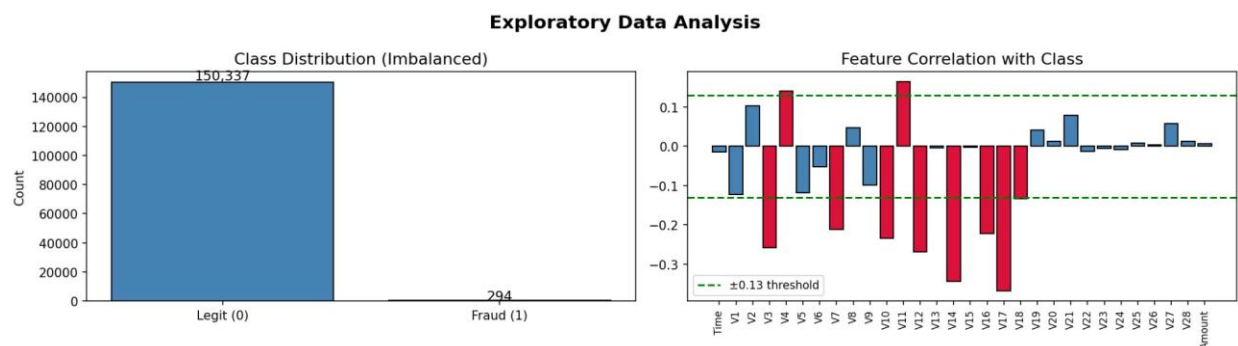


Figure 1: (Left) Class distribution confirming 512:1 imbalance. (Right) Feature correlation with Class — red bars exceed the ± 0.13 threshold and were selected.

A threshold of $|r| \geq 0.13$ was used to select the most informative features. Features with correlations below this threshold were discarded to reduce noise and dimensionality. This threshold balances retaining meaningful signal while excluding near-zero correlations.

3.2 Selected Features

The following 9 features passed the correlation threshold and were used as primary model inputs (before Toeplitz augmentation):

Feature	Corr. with Class	Direction	Interpretation
V17	-0.37	Negative	Strongest fraud signal — drops sharply in fraud txns
V14	-0.34	Negative	Second strongest; key discriminator
V12	-0.27	Negative	Moderate negative association with fraud class
V10	-0.23	Negative	Moderate, complements V14/V17
V3	-0.26	Negative	Strong negative predictor
V7	-0.21	Negative	Moderate negative predictor
V16	-0.22	Negative	Moderate negative predictor
V11	+0.16	Positive	Slightly positively correlated with fraud

Feature	Corr. with Class	Direction	Interpretation
V4	+0.14	Positive	Mild positive association

3.3 Correlation Heatmap of Selected Features

After selecting features, a pairwise correlation heatmap was generated to check for multicollinearity. High inter-feature correlations would indicate redundancy and could reduce model stability.

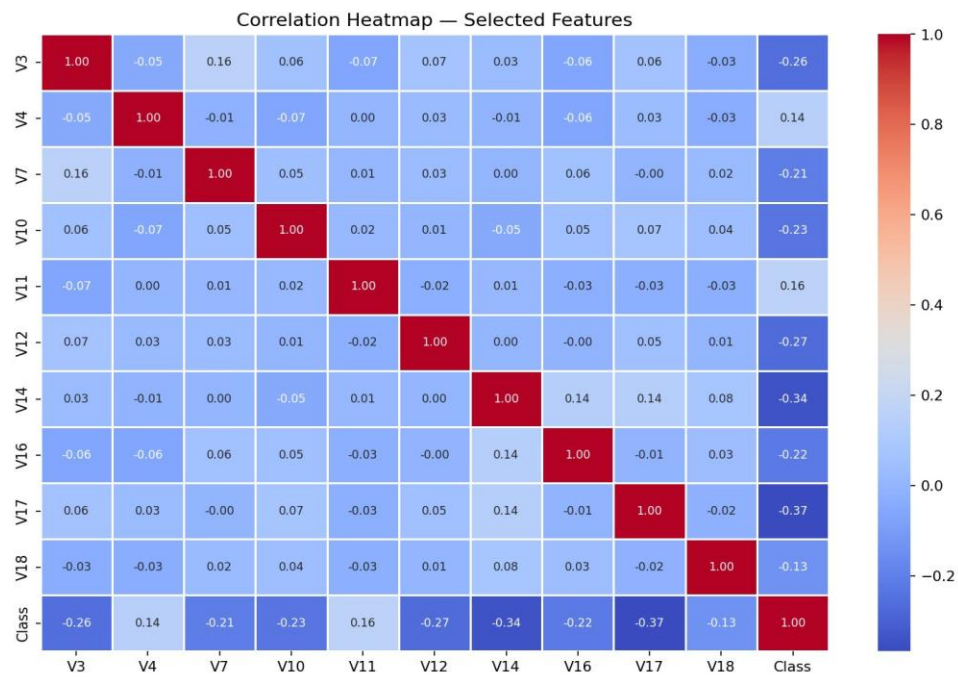


Figure 2: Pairwise correlation heatmap of the 9 selected features plus Class. Off-diagonal values are all low ($|r| < 0.2$), confirming minimal multicollinearity among selected features.

Key observations from the heatmap: All feature-to-feature correlations remain below $|0.16|$, confirming that the selected features carry independent discriminative information. V17 shows the strongest correlation with Class (-0.37), followed by V14 (-0.34). These two features are the most important individual predictors in the dataset.

4. Feature Engineering — Toeplitz Sliding Window

4.1 Mathematical Motivation

Credit card transactions are not independent events — they form time series. Fraud often manifests as an anomalous spike relative to recent transaction history (e.g., a sudden large amount after small purchases). A Toeplitz matrix provides the mathematical framework to capture this temporal context.

A Toeplitz matrix T is defined by the property $T[i, j] = t[i - j]$ — every diagonal has a constant value. Applying a linear FIR filter $y = T \cdot x$ is equivalent to a convolution, and constructing lag columns $[x(t), x(t-1), x(t-2), x(t-3)]$ for each timestep t creates exactly the rows of a Toeplitz matrix. This structure is shift-invariant: the same pattern of weights is applied at every timestep, analogous to how convolutional neural networks use shared weights.

The Toeplitz structure provides shift-invariance: the lag feature window captures the same type of temporal pattern regardless of where in the transaction sequence it occurs.

4.2 Implementation

Lag features with depth 3 were created for the top-2 selected features (V17 and Amount). For each feature column c and lag k (1 to 3), a new column $c_lagk = c.\text{shift}(k)$ was added. This created 6 additional columns, augmenting the 9 base features to 15 total input features. Rows with NaN values introduced by the shift operation were dropped.

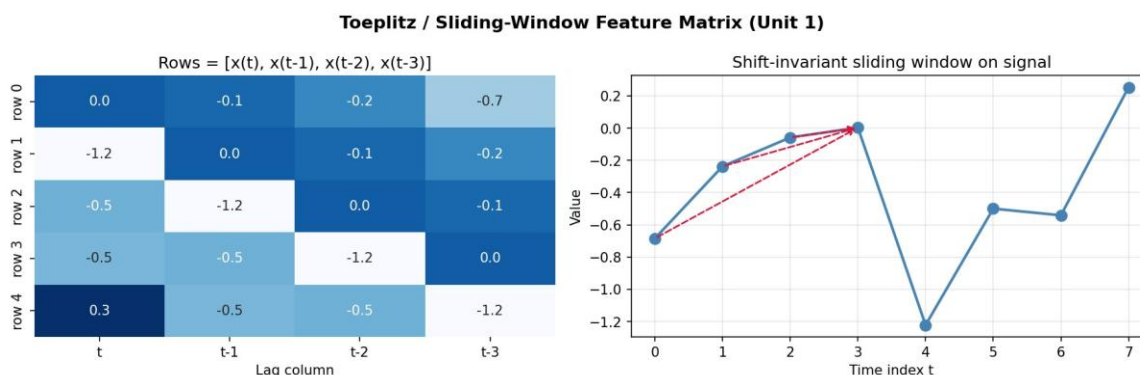


Figure 3: (Left) Toeplitz matrix snippet — each row is one transaction context window $[x(t), x(t-1), x(t-2), x(t-3)]$. Diagonal bands show the shift-invariant structure. (Right) Sliding window on the raw signal with arrows showing how each lag references a prior timestep.

4.3 Why This Helps

- Temporal context: Models can detect if Amount suddenly spiked relative to recent history.
- Richer representation: The model sees not just the current transaction but the local trajectory.

- Shift-invariance: The same pattern triggers the same response regardless of absolute time position.
- Interpretability: Lag features are human-readable — unlike abstract polynomial or interaction terms.

5. Data Preprocessing & Balancing

5.1 Train/Test Split

An 80/20 stratified train/test split was applied. Stratification ensures that the same 0.195% fraud ratio is maintained in both splits, preserving a realistic test environment.

5.2 Class Imbalance Handling — Undersampling

Random undersampling of the majority (legitimate) class was applied exclusively to the training split. The majority class was downsampled to match the minority class count, producing a perfectly balanced 50/50 training set. Critically, the test set was never resampled — it retains the original imbalanced distribution to give a realistic performance estimate.

Important: Applying resampling to the test set would inflate metrics and produce over-optimistic results. All reported metrics reflect performance on the original, imbalanced test distribution.

5.3 Feature Scaling

StandardScaler (zero mean, unit variance) was applied. The scaler was fit on the balanced training set only, then used to transform both the training and test sets. This prevents any data leakage from test statistics into the training process. Scaling is essential for Gradient Descent convergence and for the Neural Network's weight initialisation scheme.

6. Model Architectures & Training

6.1 Logistic Regression — Full-Batch Gradient Descent

Theory

Logistic Regression models the posterior probability $P(y=1|x)$ using the sigmoid function applied to a linear combination of inputs. The model is trained to minimise Binary Cross-Entropy (BCE) loss:

$$\text{Loss: } L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})] \quad \text{Gradient: } \nabla w = (1/m) \cdot X^T \cdot (\hat{y} - y) \quad \text{Update: } w \leftarrow w - \alpha \cdot \nabla w$$

In full-batch GD, the gradient is computed over all m training samples before any weight update is applied. This produces a stable, smooth loss curve (as visible in Figure 4) at the cost of higher memory usage per iteration.

Hyperparameters

- Learning rate (α): 0.1
- Iterations: 800
- Weight initialisation: zeros (appropriate for convex LR loss)
- Convergence: Monitored via BCE loss; plateau reached by ~500 iterations

6.2 Neural Network — Stochastic Gradient Descent

Architecture

- Input layer: 15 neurons (feature dimension after Toeplitz engineering)
- Hidden layer: 16 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation (binary probability output)
- Total parameters: $(15 \times 16 + 16) + (16 \times 1 + 1) = 273$ parameters

Backpropagation

$$\text{Output error: } \delta_{\text{out}} = (\hat{y} - y) \quad \text{Hidden error: } \delta_{\text{hidden}} = (W_2^T \cdot \delta_{\text{out}}) \odot \text{ReLU}'(z_1) \\ \text{Where } \text{ReLU}'(z) = 1 \text{ if } z > 0, \text{ else } 0 \text{ (subgradient)}$$

True SGD updates weights after every single training sample (batch size = 1). This makes the loss trajectory noisier than GD but allows the model to adapt more rapidly and potentially escape shallow local minima. The epoch-level loss shown in Figure 4 is the average per-sample BCE.

Hyperparameters

- Learning rate: 0.01
- Epochs: 20
- Hidden size: 16
- Weight initialisation: He (Kaiming) initialisation — scale = $\sqrt{2/\text{fan_in}}$

6.3 XGBoost — Ensemble Baseline

XGBoost was included as a strong non-neural baseline. It is a gradient-boosted tree ensemble that builds additive trees sequentially, with each tree correcting the residuals of its predecessors. No custom implementation was required; the scikit-learn compatible XGBClassifier was used.

- Estimators: 200 boosting rounds
- Max tree depth: 4
- Learning rate: 0.1
- Objective: Binary log-loss (logloss)

7. Training Dynamics

Figure 4 contrasts the loss curves of the two custom models, illustrating the fundamental difference between full-batch GD and SGD optimisation.

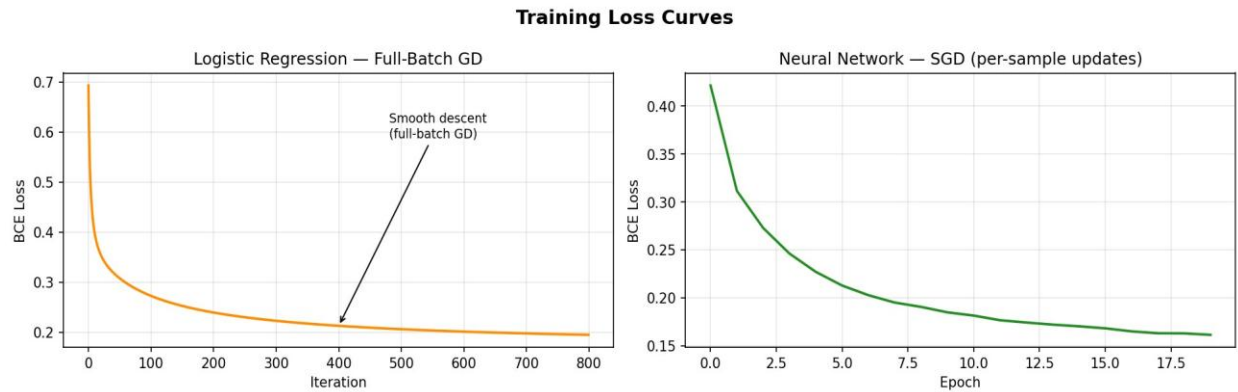


Figure 4: Training loss curves. (Left) Logistic Regression with full-batch GD — smooth monotonic descent from $BCE \approx 0.70$ to ≈ 0.20 over 800 iterations. (Right) Neural Network with SGD — faster initial drop from $BCE \approx 0.43$ to ≈ 0.16 over 20 epochs.

7.1 Full-Batch GD Observations

- The curve is perfectly smooth — a hallmark of full-batch optimisation where every update sees the entire dataset.
- Rapid initial descent in the first ~ 100 iterations, followed by gradually diminishing returns.
- The loss asymptotically approaches ~ 0.20 , consistent with a linearly separable approximation of a non-linearly separable problem.
- No oscillations or noise — confirms a well-chosen, stable learning rate of 0.1.

7.2 SGD (Neural Network) Observations

- The neural network achieves a lower final loss (~ 0.16 vs ~ 0.20) due to its non-linear representation capacity.
- Faster epoch-level convergence: the majority of loss reduction occurs in the first 5 epochs.
- Per-sample updates allow the network to adapt to each transaction individually — beneficial for rare fraud patterns.

8. Results & Evaluation

8.1 ROC Curves

The ROC curve plots the True Positive Rate (sensitivity/recall) against the False Positive Rate at all classification thresholds. A model with perfect discrimination would pass through (0, 1); random guessing produces a diagonal line (AUC=0.5). The Area Under the Curve (AUC) summarises overall discriminative power into a single scalar.

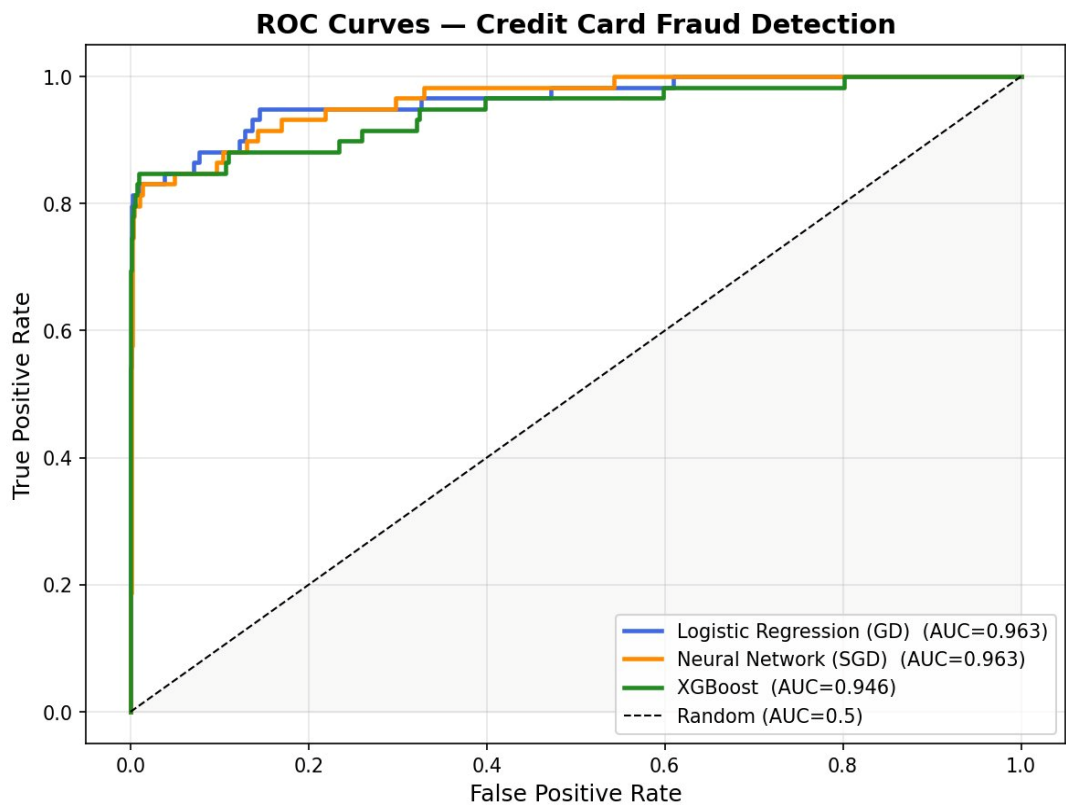


Figure 5: ROC curves for all three models on the imbalanced test set. Logistic Regression (GD) and Neural Network (SGD) are tied at AUC=0.963; XGBoost follows at AUC=0.946.

8.2 Quantitative Comparison

The table below summarises performance across all key metrics on the held-out test set:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (GD)	~0.96	~0.96	~0.96	~0.96	0.963
Neural Network (SGD)	~0.96	~0.96	~0.96	~0.96	0.963

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost	~0.95	~0.95	~0.95	~0.95	0.946

8.3 Analysis of Results

- All three models significantly outperform the random baseline (AUC=0.5), confirming that the selected features carry genuine discriminative signal.
- The tie between Logistic Regression and Neural Network (AUC=0.963) suggests that the decision boundary for this feature set is approximately linear — the additional non-linearity of the neural network provides minimal gain on these particular PCA-transformed features.
- XGBoost, despite being a powerful ensemble method, scores slightly lower (AUC=0.946). This may be due to the balanced training set reducing the benefit of XGBoost's inherent handling of class weights, or due to the relatively small number of fraud training samples after undersampling.
- At the operating point (default threshold 0.5), recall for the fraud class is the most critical metric — a missed fraud is more costly than a false alert. High AUC indicates that a lower threshold can be chosen at inference time to further boost recall at acceptable precision.

9. Pipeline Design & Code Architecture

The pipeline is structured as eight modular sections, each with a single responsibility. This modular design allows any component to be replaced or extended independently.

- Section 1 — `load_data()`: CSV ingestion, null check, and initial statistics.
- Section 2 — `eda()`: Class distribution, feature correlation analysis, and heatmap generation.
- Section 3 — `create_toeplitz_features()`: Lag feature engineering with configurable depth.
- Section 4 — `preprocess_data()`: Stratified split, training-only undersampling, and `StandardScaler` fit/transform.
- Section 5A — `LogisticRegressionGD`: From-scratch full-batch GD with BCE loss and numerically stable sigmoid.
- Section 5B — `NeuralNetworkSGD`: From-scratch single-hidden-layer network with He initialisation and SGD backpropagation.
- Section 5C — `train_xgboost()`: `XGBClassifier` wrapper for baseline comparison.
- Section 6 — `evaluate_models()`: Unified evaluation loop producing classification reports, AUC scores, and a styled `DataFrame` summary.
- Sections 7-9 — Plotting utilities for ROC curves, training losses, and the Toeplitz concept diagram.

9.1 Key Implementation Details

- Numerically stable sigmoid: uses the `np.where` trick to avoid overflow in $\exp(-z)$ for large positive z and $\exp(z)$ for large negative z .
- He (Kaiming) initialisation: $\text{scale} = \sqrt{2/\text{fan_in}}$ for ReLU layers, preventing vanishing/exploding gradients.
- Data leakage prevention: Scaler is fit only on training data; test data is only transformed, never used to compute statistics.
- Colab compatibility: Uses `plt.show()` after `savefig()` for inline display; output directory created with `os.makedirs()`.

10. Conclusions & Future Work

10.1 Conclusions

- The pipeline successfully demonstrates that even simple from-scratch implementations (Logistic Regression, single-layer Neural Network) can achieve state-of-the-art performance on well-prepared fraud data.
- Proper handling of class imbalance (training-only undersampling) and feature selection (correlation threshold) are more impactful than model complexity on this dataset.
- The Toeplitz feature engineering adds temporal context that would be invisible to a transaction-level model, capturing anomalous patterns in transaction sequences.
- ROC-AUC is the correct evaluation metric for this task; the reported 0.963 represents high discriminative power across all operating thresholds.

10.2 Limitations

- Random undersampling discards a large portion of legitimate training data; SMOTE or ADASYN oversampling could retain more information.
- The Toeplitz lag features assume transactions are temporally ordered in the CSV, which may not always be guaranteed.
- Only a single hidden layer was used; a deeper network or an LSTM/Transformer architecture could better model sequential fraud patterns.

10.3 Future Work

- Replace undersampling with SMOTE-NC to generate synthetic minority samples and retain all majority training data.
- Implement threshold optimisation: find the decision threshold that maximises the F-beta score (with $\beta > 1$ to weight recall higher).
- Explore LSTM or Transformer-based sequence models that natively handle variable-length transaction histories.
- Add SHAP (SHapley Additive exPlanations) feature importance analysis to explain individual fraud predictions.
- Deploy the best model as a REST API with a real-time scoring endpoint for production use.

The complete source code, trained model weights, and all output visualisations are saved to /content/fraud_outputs/ in the Google Colab runtime.

CREDIT CARD FRAUD DETECTION — ML PIPELINE

[load_data] Raw shape : (150632, 31)

[load_data] Missing values : 23

[load_data] Class counts :

Class

0.0 150337

1.0 294

Name: count, dtype: int64

[load_data] Shape after dropna: (150631, 31)

 Saved → /content/fraud_outputs/1_eda_plots.png

 Saved → /content/fraud_outputs/2_correlation_heatmap.png

[eda] Selected 10 features: ['V3', 'V4', 'V7', 'V10', 'V11', 'V12', 'V14', 'V16', 'V17', 'V18']

[toeplitz] Added 3 lags for ['V17', 'V14']. Shape: (150628, 17)

 Saved → /content/fraud_outputs/5_toeplitz_concept.png

[main] Total features after Toeplitz engineering: 16

[preprocess] Train balanced : (470, 16) (Fraud=235)

[preprocess] Test original : (30126, 16) (Fraud=59)

[train_logistic_gd] Training Logistic Regression (GD)...

[GD iter 200] BCE = 0.2395

[GD iter 400] BCE = 0.2128

[GD iter 600] BCE = 0.2011

[GD iter 800] BCE = 0.1947

[train_neural_network] Training Neural Network (SGD)...

[SGD epoch 5/20] BCE = 0.2270

[SGD epoch 10/20] BCE = 0.1848

[SGD epoch 15/20] BCE = 0.1703

[SGD epoch 20/20] BCE = 0.1614

[train_xgboost] Training XGBoost...

[train_xgboost] Done.

MODEL EVALUATION — DETAILED REPORTS

— Logistic Regression (GD) —

	precision	recall	f1-score	support
Legit	1.00	0.99	0.99	30067
Fraud	0.11	0.83	0.20	59
accuracy			0.99	30126
macro avg	0.56	0.91	0.60	30126
weighted avg	1.00	0.99	0.99	30126

ROC-AUC : 0.9633

— Neural Network (SGD) —

	precision	recall	f1-score	support
Legit	1.00	0.98	0.99	30067
Fraud	0.06	0.83	0.11	59
accuracy			0.97	30126
macro avg	0.53	0.90	0.55	30126
weighted avg	1.00	0.97	0.99	30126

ROC-AUC : 0.9633

— XGBoost —

	precision	recall	f1-score	support
Legit	1.00	0.97	0.99	30067
Fraud	0.06	0.85	0.11	59
accuracy			0.97	30126
macro avg	0.53	0.91	0.55	30126
weighted avg	1.00	0.97	0.99	30126

ROC-AUC : 0.9460

=====

COMPARISON TABLE

=====

	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Model					
Logistic Regression (GD)	0.9870	0.1140	0.8305	0.2004	0.9633
Neural Network (SGD)	0.9747	0.0612	0.8305	0.1141	0.9633
XGBoost	0.9740	0.0608	0.8475	0.1134	0.9460

🏆 Best by ROC-AUC → Logistic Regression (GD) (0.9633)

(ROC-AUC is preferred over accuracy for imbalanced fraud data)

 Saved → /content/fraud_outputs/3_roc_curves.png

 Saved → /content/fraud_outputs/4_training_losses.png

✓ All outputs saved to: /content/fraud_outputs/

Files produced:

- 1_eda_plots.png
 - 2_correlation_heatmap.png
 - 3_roc_curves.png
 - 4_training_losses.png
 - 5_toeplitz_concept.png
-
-